b38b5a11-18ac-4578-a037-3027a04df3ea

<prev_user>

<request>

You are working on an issue in the 'neynarxyz/nodejs-sdk' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/nodejs-sdk/nodejs-sdk (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-04T04:35:54.921Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section details on the original issue you should resolve

<issue_title>The link to the getting started guide is outdated</issue_title>
<issue_description>Can't access the getting started guide for the SDK (also seems like release tags are outdated, since the published npm version differs from the latest release tag in github)</issue_description>

<agent_instructions>Ignore the getting started guide link, that's fixed already.

fix the publish to npm version mismatch by publishing a new release on github with generated release notes in the publish to npm workflow after an npm publish is successful. use the version to name the release accordingly</agent_instructions>

Comments on the Issue (you are @copilot in this section)

<comments>
</comments>

</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

Excellent! Let me create a comprehensive summary:

Summary

I've successfully fixed the npm version mismatch issue by modifying the GitHub Actions workflow to automatically create a GitHub release after publishing to npm.

What Was Changed:

File Modified: `.github/workflows/publish-to-npm.yaml`

Changes Made (16 lines added):
1. **Extract version from package.json**: Added a step that reads the version from `package.json` and stores it as an output variable
2. **Create GitHub Release**: Added a step that:
   - Checks if a release with the version tag already exists
   - Creates a new release with the version tag (e.g., `v3.77.0`) if it doesn't exist
   - Uses `--generate-notes` flag to automatically generate release notes from commits
   - Creates the git tag automatically as part of the release creation

How It Works:

The workflow is triggered when `package.json` changes on the main branch:
1. Builds the project
2. Publishes to npm

3. **NEW**: Extracts the version from package.json
4. **NEW**: Creates a GitHub release with auto-generated notes

Benefits:

- **Solves the version mismatch**: Every npm publish will now have a corresponding GitHub release
- **Automatic release notes**: Uses GitHub's built-in release notes generation
- **Error handling**: Won't fail if a release already exists
- **Minimal changes**: Only 16 lines added to the existing workflow
- **No security issues**: Passed CodeQL security scan with 0 alerts

The next time `package.json` is updated and pushed to main, the workflow will automatically create a GitHub release matching the npm version!

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'neynarxyz/nodejs-sdk' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/nodejs-sdk/nodejs-sdk (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-04T04:51:42.447Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Auto-create GitHub releases on npm publish

````
</pr_title>

<pr_description>
npm versions were ahead of GitHub releases because the publish
workflow only pushed to npm.

## Changes

Modified `.github/workflows/publish-to-npm.yaml` to create
GitHub releases post-publish:

- Extract version from `package.json` after successful npm
publish
- Create release with tag `v{version}` using `gh release create
--generate-notes`
- Skip if release already exists to handle edge cases

```yaml
- name: Extract version from package.json
  id: package-version
  run: echo "version=$(node -p "require('./
package.json').version")" >> $GITHUB_OUTPUT

- name: Create GitHub Release
  run: |
    if gh release view "v${{ steps.package-
version.outputs.version }}" &> /dev/null; then
      echo "Release v${{ steps.package-
version.outputs.version }} already exists"
    else
      gh release create "v${{ steps.package-
version.outputs.version }}" \
        --title "v${{ steps.package-version.outputs.version }}"
\
        --generate-notes
    fi
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Future package.json updates will automatically create matching
GitHub releases with auto-generated notes from commits.

<issue_title>The link to the getting started guide is outdated</
issue_title>
&gt;<issue_description>Can't access the getting started guide
for the SDK (also seems like release tags are outdated, since
````

```
the published npm version differs from the latest release tag in
github)</issue_description>
&gt;
&gt;<agent_instructions>Ignore the getting started guide link,
that's fixed already.
&gt;
&gt; fix the publish to npm version mismatch by publishing a new
release on github with generated release notes in the publish to
npm workflow after an npm publish is successful. use the version
to name the release accordingly</agent_instructions>
&gt;
&gt; ## Comments on the Issue (you are @copilot in this section)
&gt;
&gt;<comments>
&gt;</comments>
&gt;

- Fixes neynarxyz/nodejs-sdk#379
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>
  <comment_new>
  <comment_id>3483825428</comment_id>
  <author>@manan19</author>
  @copilot give me instructions on how to set up the
  GITHUB_TOKEN?
  </comment_new>
</pr_comments>

</comments>

----
The last **3** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
```

```
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```